

1. Backend Engineer Contract / SOW

Backend Engineer Contract Scope of Work

Document Title: Backend Engineer Contract Scope of Work

Project: ValidDs

Contractor: Johnpaul Laniran

Company: Supreme Ecom

Start Date: 03/23/2026

Initial Contract Term: 30-day milestone period, with continuation up to 5–6 months total subject to successful delivery, documentation quality, and mutual agreement

Compensation: \$9,500

Payment Cadence: Monthly

Reporting To: Ugo Orakwe

Work Arrangement: Independent contractor / freelance engagement

1. Purpose of Engagement

The Contractor is being engaged to serve as the backend engineer for the ValidDs product build. ValidDs is a creative-led product research and validation platform for dropshippers. The backend function is responsible for establishing the data foundation, ingestion paths, schema structure, core APIs, validation signal infrastructure, and system reliability required to support the V1 product.

This engagement begins with a 30-day milestone period intended to validate execution quality, delivery reliability, documentation quality, and architectural fit for the project. Continuation beyond the first 30 days depends on successful completion of the agreed milestone outputs and alignment with the company on the next phase of work.

2. Scope of Work

During the engagement, the Contractor will be responsible for backend-related work required to move ValidDs toward V1 launch readiness, including:

A. Data Acquisition Foundation

- evaluate and select the fastest workable TikTok data acquisition path for V1
- identify and document fallback options if the primary path becomes unstable
- establish the initial ingestion approach required for product, video, and trend data

B. Data Modeling and Schema Design

- define and implement the initial database schema for:
 - products
 - videos
 - trend records
 - competitor / store references
 - supplier / sourceability references
- ensure the data model supports product feed, product detail, filtering, and validation outputs

C. Backend Architecture

- define the backend system architecture for V1
- establish repository structure, service structure, and environment setup
- make implementation choices that support fast delivery without unnecessary complexity

D. API and Product Data Delivery

- create backend endpoints required for:
 - product feed
 - product detail
 - signal / trend / freshness data
- support frontend consumption with clean, documented payloads

E. Reliability, Monitoring, and Fallback Behavior

- implement monitoring and logging for core backend functions
- establish data freshness and fallback handling so the product remains usable if the data source becomes unstable
- clearly document system risks and operational constraints

F. Documentation

- document architecture, schemas, endpoint contracts, environment setup, known risks, and next-step recommendations
- maintain documentation in a way that allows continuity if additional engineers are added later

3. Deliverables During Initial 30-Day Milestone Period

The following items constitute the required milestone outputs for the first 30 days of the engagement.

Deliverable 1: Backend Architecture Draft

A written architecture document covering:

- backend stack
- ingestion path
- storage approach
- monitoring approach
- fallback strategy
- key system assumptions and risks

Deliverable 2: Initial Data Acquisition Path

A working first-path approach for acquiring and storing the core V1 data needed to support the product feed and product detail surfaces, including documentation of fallback options.

Deliverable 3: Core Schema and Data Model

An implemented and documented schema covering the core product entities needed for V1.

Deliverable 4: Product Feed Endpoint

A first usable backend endpoint that returns product feed data in a frontend-consumable format.

Deliverable 5: Product Detail Endpoint

A first usable backend endpoint that returns the core product detail payload, including trend / validation-supporting data fields.

Deliverable 6: Freshness / Failure Handling

Implementation of:

- last updated logic
- stale-data handling
- fallback behavior when fresh data is unavailable

Deliverable 7: Documentation Pack

Current documentation covering:

- setup
- architecture
- schema
- endpoints
- known issues
- next-step priorities

4. Initial 30-Day Continuation Gate

This contract begins with a 30-day milestone period. At the end of that period, the company will assess whether to continue the engagement into the remaining contract term.

Continuation will be based on:

- quality of architecture decisions
- actual delivery against agreed outputs
- clarity and completeness of documentation
- communication and responsiveness

- ability to work within project scope and timeline
- fit with the execution needs of the ValidDs build

If the company determines that continuation is not in the best interest of the project after the 30-day review, the engagement may conclude at the end of the initial milestone period. The Contractor will still be paid for work completed through the effective end date of the engagement, in accordance with the agreed payment terms.

5. Working Expectations

The Contractor agrees to:

- maintain consistent progress during the engagement
- communicate blockers early
- work in sprint-based execution cycles
- participate in scheduled check-ins and review sessions
- keep project documentation updated
- avoid unnecessary overengineering that slows V1 progress
- align implementation with the locked V1 scope and priorities

Expected minimum weekly availability: **45 hours per week**

The Contractor agrees to be reasonably reachable during working periods and responsive to urgent project matters where feasible.

6. Communication Cadence

The Contractor will work under a structured weekly cadence defined by the Product Manager. This includes:

- weekly planning
- midweek progress visibility
- end-of-week output review
- clear ownership of committed deliverables

7. Ownership of Work Product

All work product created during this engagement in connection with ValidDs, including code, documentation, specifications, architecture decisions, and related materials, shall be the property of Supreme Ecom upon payment for the work performed, subject to any standard company contractor agreement terms.

8. Confidentiality

The Contractor acknowledges that all product, business, technical, and strategic information shared during the engagement is confidential and may not be disclosed or used outside the scope of the engagement.

9. Contract Continuation Beyond 30 Days

If the Contractor successfully completes the initial milestone period and the company elects to continue the engagement, the remaining contract term will focus on continued backend buildout toward V1 launch, including expanded validation logic, reliability improvements, additional product data support, and other backend priorities defined by the company.

10. Acceptance

Contractor Name: Laniran Johnpaul

Signature: 

Date: 21/03/2026

Company Representative Name: Ugo Orakwe

Signature: Ugo Orakwe

Date: 03/20/2026

2. 30-Day Success Plan

Overall 30-Day Goal

At the end of 30 days, ValidDs must have a usable backend foundation that allows the product to begin real build execution, not just technical exploration.

That means by day 30:

- the backend direction is chosen
- the system structure exists
- data is flowing through at least one workable path
- the core product entities are established
- the frontend can begin consuming real backend outputs
- the company is not guessing what backend is doing

Week 1 - Architecture and Data Acquisition Decision

Week 1 objective

Remove ambiguity around the backend direction and get the foundation set.

Required outputs by end of Week 1

1. Backend architecture draft

- stack
- service structure
- storage approach
- hosting / environment assumptions
- monitoring approach
- fallback philosophy

2. Data acquisition strategy memo

- primary TikTok data acquisition path
- 2–3 fallback options
- pros / cons of each
- V1 recommendation

3. Initial schema draft

- products
- videos
- trends
- competitor / store references
- supplier / sourceability references

4. Repo and environment setup

- repository ready
- base project structure
- environment variable structure
- initial setup instructions

Week 1 success criteria

- no broad confusion remains about backend direction
- no “still researching everything” excuse at end of week
- architecture and acquisition path are concrete enough to build on

Week 2 - First Working Data Path

Week 2 objective

Get raw data into a structured backend shape.

Required outputs by end of Week 2

1. Primary acquisition proof of concept

- first workable path pulling relevant data
- not necessarily perfect, but usable

2. Normalized product record shape

- raw incoming data transformed into a product-oriented structure

3. Data storage working in dev environment

- initial records stored and retrievable

4. Logging / monitoring baseline

- failures and job issues visible
- basic alerts or visibility defined

5. Documented known risks

- data access risk
- fallback risk
- competitor/store data limitations
- what is being deferred

Week 2 success criteria

- data is no longer theoretical
- backend has a real input path
- the team can inspect real stored records

Week 3 - Product-Usable Backend Outputs

Week 3 objective

Make backend outputs usable for product surfaces.

Required outputs by end of Week 3

1. First product feed endpoint

- returns usable list data for frontend

2. First product detail endpoint

- returns usable detail data for one product

3. Initial signal-supporting fields included

- trend support
- freshness
- engagement / validation-supporting data
- store / supplier support fields where available

4. Freshness logic

- last updated field
- stale-data behavior

5. Backend documentation updated

- endpoint structure
- field definitions
- assumptions

Week 3 success criteria

- frontend can begin integrating against real outputs
- product feed and detail are no longer abstract
- data trust handling has started

Week 4 - Stability, Documentation, and Review Gate

Week 4 objective

Prove the backend is ready to continue into the next build phase.

Required outputs by end of Week 4

- 1. Architecture and setup documentation complete**
- 2. Core schema finalized for V1 phase 1**
- 3. Feed and detail endpoints stable enough for continued frontend integration**
- 4. Freshness / fallback handling functioning**
- 5. Month 2 backend roadmap**
 - what comes next
 - what remains risky
 - what should be cut or delayed
- 6. 30-day review package**
 - what was delivered
 - what remains blocked
 - quality of documentation
 - recommendation for continuation

Week 4 success criteria

- backend has delivered visible progress
- documentation is strong enough for continuity
- leadership can decide continuation based on facts

3. Weekly Cadence

Monday - Weekly Planning Call (30-45 min)

Purpose:

- confirm weekly goals
- confirm what must ship by Friday
- align on blockers, decisions, and dependencies

Monday agenda

1. Review last week's completed outputs
2. Confirm this week's exact deliverables
3. Confirm any blocked decisions needed from you
4. Confirm whether current work still aligns with locked V1
5. Restate Friday review expectations

Wednesday - Midweek Async Update

Purpose:

- keep visibility high without wasting meeting time

Required update format

You send:

1. what is completed so far
2. what is in progress
3. what is blocked
4. any decision needed from me

5. whether Friday deliverables are still on track

Friday - Output Review (30-45 min)

Purpose:

- review what actually got produced
- compare promised output vs actual output
- decide what rolls forward

Friday review agenda

1. Show actual output
 - docs
 - schema
 - repo updates
 - endpoints
 - monitoring
2. Confirm what is production-useful vs exploratory
3. Note any risks / changes
4. Capture what becomes next week's carryover
5. Confirm updated documentation status

Ad hoc communication rule

We use async messaging for:

- blockers and quick questions
- dependency clarification
- urgent risk alerts

4. Backend Documentation Requirement

A. System Overview

Must explain:

- what the backend system currently does
- what data source path is being used
- what the major system components are
- where the biggest risks are

B. Data Acquisition Documentation

Must include:

- primary acquisition path
- fallback acquisition paths
- tools / services / libraries used
- rate limits / reliability concerns if known
- how often data runs
- what happens when acquisition fails

C. Schema Documentation

For each major entity, document:

- entity name
- purpose
- fields
- relationships
- any important assumptions

At minimum:

- products
- videos
- trends
- competitor / stores
- supplier / sourceability

D. Endpoint Documentation

For every endpoint built, document:

- endpoint name
- purpose
- request format
- response format
- important fields
- known limitations

E. Environment / Setup Documentation

Must include:

- required services
- environment variables
- how to run the project locally
- how jobs or background processes are started
- deployment assumptions

F. Monitoring / Failure Handling Documentation

Must include:

- what is monitored

- how failures are detected
- what stale-data behavior exists
- who gets alerted
- what fallback behavior exists

G. Open Risks / Known Gaps

Must include:

- current unresolved issues
- what is intentionally deferred
- where data quality is weak
- what depends on later decisions

H. Change Log / Decision Log

Should record:

- key backend decisions made
- why they were made
- what was rejected
- what might change later

Documentation standard

Documentation must be:

- current and readable by another engineer
- sufficient for continuity if someone else joins